

TP1: Demostración

Paradigmas de Programación

Grupo: typeError

0.1. Propiedad a demostrar

Toda expresión tiene un literal más que su cantidad de operadores

$$\forall e :: \text{Expr} \text{ cantLit } e = S(\text{cantOp } e)$$

1. Predicado para inducción estructural

La inducción estructural se hará sobre la estructura de Expr y el predicado unario es:

$$P(e) \equiv \text{cantLit}(e) = S(\text{cantOp}(e))$$

2. Esquema formal de inducción estructural

La estructura inductiva es **Expr**, cuyos constructores son: Const, Rango, Suma, Resta, Mult, Div.

Vamos a demostrar lo siguiente:

$$P(e) \forall e :: \text{Expr}$$

Esto lo haremos dando uso de la técnica de inducción estructural. Donde para el caso base tomaremos los constructores no recursivos de la definición y demostraremos que el predicado vale para toda variable que acompañe estos constructores, es decir:

1. Casos base:

- $P(\text{Const } x) \forall x :: \text{Float}$
- $P(\text{Rango } a \ b) \forall a, b :: \text{Float}$

2. Casos inductivos:

- $\forall a, b :: \text{Expr}. P(a) \wedge P(b) \Rightarrow P(\text{Suma } a \ b)$
- $\forall a, b :: \text{Expr}. P(a) \wedge P(b) \Rightarrow P(\text{Resta } a \ b)$
- $\forall a, b :: \text{Expr}. P(a) \wedge P(b) \Rightarrow P(\text{Mult } a \ b)$
- $\forall a, b :: \text{Expr}. P(a) \wedge P(b) \Rightarrow P(\text{Div } a \ b)$

3. Tendremos 4 hipótesis inductivas, una para cada constructor recursivo, estas son:

- $\{\text{HIS}\} \equiv ((P(a) \wedge P(b)) \Rightarrow P(\text{Suma } a \ b)), \forall a, b :: \text{Expr}$
- $\{\text{HIR}\} \equiv ((P(a) \wedge P(b)) \Rightarrow P(\text{Resta } a \ b)), \forall a, b :: \text{Expr}$
- $\{\text{HIM}\} \equiv ((P(a) \wedge P(b)) \Rightarrow P(\text{Mult } a \ b)), \forall a, b :: \text{Expr}$
- $\{\text{HID}\} \equiv ((P(a) \wedge P(b)) \Rightarrow P(\text{Div } a \ b)), \forall a, b :: \text{Expr}$

Al estar definidas como una implicación vamos a aprovecharnos de que si nosotros queremos demostrar el consecuente podemos asumir el precedente como verdadero, y esto significa que si queremos demostrar que P vale para un caso recursivo podemos asumir que P vale para las subexpresiones recursivas del caso.

3. Demostración

3.1. Caso base: Const

Se quiere probar:

$$\text{cantLit}(\text{Const } x) = S(\text{cantOp}(\text{Const } x)), \forall x :: \text{Float}$$

Se aplican las definiciones:

- {L1}: $\text{cantLit}(\text{Const } x) = SZ$
- {O1}: $\text{cantOp}(\text{Const } x) = Z$

Por lo tanto, la expresión quedará como:

$$\text{cantLit}(\text{Const } x) = SZ = S(\text{cantOp}(\text{Const } x))$$

Tautología

□

3.2. Caso base: Rango

Se quiere probar:

$$\text{cantLit}(\text{Rango } a \text{ } b) = S(\text{cantOp}(\text{Const } a \text{ } b)), \forall a, b :: \text{Float}$$

Se aplican las definiciones:

- {L2}: $\text{cantLit}(\text{Rango } a \text{ } b) = SZ$
- {O2}: $\text{cantOp}(\text{Rango } a \text{ } b) = Z$

Por lo tanto, la expresión quedará como:

$$\text{cantLit}(\text{Rango } a \text{ } b) = SZ = S(\text{cantOp}(\text{Rango } a \text{ } b))$$

Tautología

□

3.3. Caso inductivo: Suma

Ahora para demostrar el caso inductivo de la propiedad debemos demostrar P para el resto de constructores del tipo de dato, pero por similitud de demostraciones y simplicidad del enunciado solo demostraremos P para el constructor Suma:

La hipótesis inductiva ($\{HI\}$) será:

$$P(a) : \text{cantLit}(a) = S(\text{cantOp}(a))$$

$$P(b) : \text{cantLit}(b) = S(\text{cantOp}(b))$$

Se quiere demostrar

$$P(\text{Suma } a \text{ } b) : \text{cantLit}(\text{Suma } a \text{ } b) = S(\text{cantOp}(\text{Suma } a \text{ } b)), \forall a, b :: \text{Expr}$$

lo cual puede hacerse de la siguiente forma:

$$\begin{array}{lll}
 & \text{cantLit}(\text{Suma } a \text{ } b) \equiv S(\text{cantOp}(\text{Suma } a \text{ } b)) & \\
 \{L3\} & \text{suma}(\text{cantLit}(a) \text{ } \text{cantLit}(b)) \equiv S(S(\text{suma}(\text{cantOp}(a) \text{ } \text{cantOp}(b)))) & \{O3\} \\
 \{HI\} & \text{suma}(S(\text{cantOp}(a)) \text{ } S(\text{cantOp}(b))) \equiv S(S(\text{suma}(\text{cantOp}(a) \text{ } \text{cantOp}(b)))) & \\
 \{S2\} & S(\text{suma}(\text{cantOp}(a) \text{ } S(\text{cantOp}(b)))) \equiv S(S(\text{suma}(\text{cantOp}(a) \text{ } \text{cantOp}(b)))) & \\
 \{CONMUT\} & S(\text{suma}(S(\text{cantOp}(b)) \text{ } \text{cantOp}(a))) \equiv S(S(\text{suma}(\text{cantOp}(a) \text{ } \text{cantOp}(b)))) & \\
 \{S2\} & S(S(\text{suma}(\text{cantOp}(b) \text{ } \text{cantOp}(a)))) \equiv S(S(\text{suma}(\text{cantOp}(a) \text{ } \text{cantOp}(b)))) & \\
 \{CONMUT\} & S(S(\text{suma}(\text{cantOp}(a) \text{ } \text{cantOp}(b)))) \equiv S(S(\text{suma}(\text{cantOp}(a) \text{ } \text{cantOp}(b)))) & \\
 & \textbf{Tautología} &
 \end{array}$$

Por lo tanto, queda demostrado P para los constructores que se nos pidió y como la demostración para el resto de constructores es similar a la que dimos, podemos asumir:

$$P(e) \text{ vale } \forall e :: \text{Expr}$$

□